Universidad Autónoma de Nuevo León

Facultad de ingeniería, mecánica y eléctrica

Sistemas Operativos

Actividad Fundamental 3

Fotografía	Matrícula	Nombre	Carrera	Participación
	2070009	Cruz Ortega Angely Ximena	ITS	**100%**
	2037907	García Martínez Ari Sebastián	IAS	**100%**
	2099212	Pérez Barrón Diego Alonso	ITS	**100%**

Dra. Norma Edith Marín Martínez

Enero-Junio 2026 San Nicolás de los Garza, NL

Universidad Autónoma de Nuevo León

Facultad de ingeniería, mecánica y eléctrica

Sistemas Operativos

Actividad Fundamental 2

Escribir las conclusiones individuales a mano

OK 24-03-2026

Falta numeración en las hojas

Fotografía	Matrícula	Nombre	Carrera	Participación
	2070009	Cruz Ortega Angely Ximena	ITS	**100%**
	2037907	García Martínez Ari Sebastián	IAS	**100%**
	2099212	Pérez Barrón Diego Alonso	ITS	**100%**

Dr. Norma Edith Marín Martínez

Enero-Junio 2026 San Nicolás de los Garza, NL

CamScanner

Tabla de contenido

Introducción 5
Gestión de memoria y sistema de archivos 5
En computadoras 6
Microsoft Windows 6
Linux (Ubuntu, Fedora, Debian, Arch) 7
macOS (Apple) 7
En dispositivos móviles 8
Android 9
iOS (Apple) 10
HarmonyOS (Huawei) 10
Sistemas operativos de red 12
Windows Server 12
Linux Server (Ubuntu Server, RHEL, Debian) 13
Importancia de administrador de memoria 14
Fundamentos de la Importancia 14
Consecuencias de una Mala Gestión de Memoria 15
Llevado de control de los espacios disponibles y ocupados 16
Representación del Espacio Libre y Ocupado 16
Asignación de Memoria 16
Paginación 17
Segmentación 17
Liberación y Algoritmos de Reemplazo de Páginas 18
Problemas más frecuentes en la administración de memoria 19
Fragmentación 19
Fuga de Memoria (Memory Leak) 19
Thrashing (Hiperpaginación) 20
Deadlock por Recursos de Memoria 20
Desbordamiento de Búfer (Buffer Overflow) 21
Use-After-Free 21
Conclusiones individuales 22
Angely Ximena Cruz Ortega 22
Ari Sebastián García Martínez 25

Diego Alonso Pérez Barrón 27
Referencias 29

TIPOS DE MEMORIA

Introducción

La administración de memoria es uno de los pilares fundamentales del diseño de sistemas operativos modernos. Independientemente de si se trata de una computadora de escritorio, un teléfono inteligente o un servidor empresarial, el sistema operativo tiene la responsabilidad de coordinar el uso de un recurso escaso y costoso: la memoria principal. Una gestión ineficiente puede derivar en pérdida de rendimiento, vulnerabilidades de seguridad o incluso la caída completa del sistema.

Esta investigación aborda de manera sistemática los tipos de memoria presentes en distintas plataformas, los sistemas de archivos que cada entorno utiliza, la importancia crítica de administrar correctamente la memoria, las técnicas que emplean los sistemas operativos para controlar el espacio disponible y ocupado, así como los problemas más frecuentes que surgen cuando dicha administración falla. El análisis comprende tres categorías de entornos: computadoras personales (Windows, Linux y macOS), dispositivos móviles (Android, iOS y HarmonyOS) y sistemas operativos de red (Windows Server y Linux Server).

Gestión de memoria y sistema de archivos

Antes de analizar plataforma por plataforma, es necesario comprender la jerarquía general de memoria que existe en cualquier sistema informático. Esta jerarquía organiza los distintos tipos de almacenamiento desde el más rápido y costoso hasta el más lento y económico:

Tipo de Memoria	Velocidad	Capacidad Típica	Función Principal	Volátil
Registros del CPU	< 1 ns	Bytes (8–64 registros)	Operandos inmediatos del procesador	Sí
Caché L1	1–4 ns	32–64 KB por núcleo	Instrucciones y datos más recientes	Sí
Caché L2	4–12 ns	256 KB – 1 MB	Desbordamiento de L1	Sí
Caché L3	30–50 ns	8–64 MB (compartido)	Compartido entre núcleos del CPU	Sí

RAM (DRAM)	50–100 ns	8–128 GB	Espacio de trabajo de procesos activos	Sí
ROM / UEFI / BIOS	~100 ns	16–64 MB	Firmware de arranque y POST	No
Memoria Virtual	1–10 ms (SSD)	Limitada por disco	Extensión lógica de la RAM sobre almacenamiento secundario	No

Tabla 1. Jerarquía general de tipos de memoria en sistemas informáticos. La velocidad y el costo disminuyen conforme se desciende en la jerarquía, mientras que la capacidad aumenta.

En computadoras

Las computadoras personales son el entorno donde la administración de memoria alcanza mayor complejidad visible para el usuario. Los tres sistemas operativos más representativos —Windows, Linux y macOS— presentan enfoques distintos aunque complementarios.

Microsoft Windows

Windows gestiona la memoria a través del componente ntoskrnl.exe (el núcleo del sistema operativo), que implementa paginación bajo demanda. Esto significa que las páginas de un proceso se cargan en RAM únicamente cuando se accede a ellas, no en el momento en que el proceso es lanzado. Cada página tiene un tamaño estándar de 4 KB, aunque Windows soporta páginas grandes de 2 MB (Large Pages) para procesos de alto rendimiento como bases de datos.

El conjunto de páginas físicas asignadas actualmente a un proceso se denomina Working Set. Cuando la RAM se satura, el Administrador de Memoria de Windows mueve páginas del Working Set hacia el archivo de paginación pagefile.sys, ubicado en el disco. Este proceso se denomina swapping o paginación secundaria. En sistemas de 64 bits, cada proceso puede disponer de hasta 2 TB de espacio de direcciones virtual, aunque la RAM física sea mucho menor.

Sistema de archivos — NTFS: El sistema de archivos principal de Windows es NTFS (New Technology File System). Su estructura central es la Tabla Maestra de Archivos (MFT), que almacena un registro de cada archivo y carpeta del volumen, incluyendo sus atributos, permisos y ubicación en disco. NTFS implementa journaling: antes de realizar cualquier modificación, registra la operación en un diario (log), lo que permite recuperar la consistencia del sistema de archivos tras un corte de energía sin necesidad de escanear todo el disco. Adicionalmente, NTFS soporta listas de control de acceso (ACL), compresión nativa y cifrado a nivel de archivo mediante EFS (Encrypting File System).

Linux (Ubuntu, Fedora, Debian, Arch)

El kernel de Linux utiliza el algoritmo Buddy System para la asignación de memoria física. Este mecanismo divide la memoria en bloques de potencias de dos (1, 2, 4, 8... páginas), lo que facilita la unión de bloques libres adyacentes y reduce la fragmentación. Para objetos del kernel de tamaño pequeño y frecuentemente reutilizados (como descriptores de procesos o inodos), Linux emplea el Slab Allocator, que mantiene cachés de objetos ya inicializados y los reutiliza sin necesidad de reinicializarlos.

Un comportamiento notable de Linux es el uso agresivo de la memoria libre para caché de páginas (Page Cache). Toda la RAM que no está asignada a procesos activos se usa para almacenar bloques de archivos recientemente leídos del disco, acelerando lecturas futuras. Cuando la presión de memoria aumenta, el kernel puede desalojar estas páginas para cederlas a procesos que las necesiten. Si la memoria escasea de manera crítica, el OOM Killer (Out-of-Memory Killer) termina el proceso que mayor cantidad de memoria consume para estabilizar el sistema.

El parámetro vm.swappiness (rango 0–100) controla la tendencia del kernel a usar la partición swap antes de desalojar páginas de caché. Un valor de 10 indica que el kernel prefiere mantener el caché en RAM el mayor tiempo posible; un valor de 100 lo hace más agresivo en el uso de swap.

Sistemas de archivos — ext4, btrfs, XFS: El sistema de archivos más común en distribuciones Linux es ext4. Utiliza extents (bloques contiguos) para reducir la fragmentación en archivos grandes, implementa journaling completo y admite volúmenes de hasta 1 EB. btrfs (B-tree File System) es la alternativa moderna: utiliza Copy-on-Write (CoW), lo que significa que las modificaciones no sobreescriben los datos originales sino que crean una nueva copia, permitiendo snapshots instantáneos y recuperación ante fallos. XFS es preferido en entornos de servidor por su excelente rendimiento en archivos de gran tamaño y alta concurrencia. El acceso a todos estos sistemas de archivos se realiza a través de la capa VFS (Virtual File System), que actúa como interfaz unificada independientemente del sistema de archivos subyacente.

macOS (Apple)

macOS se basa en el kernel XNU (una combinación de Mach y partes de BSD). La característica más relevante en los equipos con chip Apple Silicon (M1, M2, M3) es la Unified Memory Architecture (UMA): la CPU, la GPU, el Neural Engine y el motor de señales comparten un único pool de memoria de alta velocidad integrado en el mismo chip (SoC). Esto elimina la necesidad de copiar datos entre memorias separadas, reduciendo la latencia de operaciones gráficas e IA de manera significativa.

Antes de recurrir al swap en disco, macOS utiliza un Memory Compressor: las páginas inactivas se comprimen en RAM con ratios típicos de 2:1 a 3:1, liberando espacio sin acceder al almacenamiento secundario. Solo cuando la compresión no es suficiente, las páginas se mueven al archivo de swap en el SSD.

Sistema de archivos — APFS: Apple File System (APFS), introducido en 2017, es el sistema de archivos predeterminado en todos los dispositivos Apple. Al igual que btrfs, utiliza Copy-on-Write: las escrituras nunca sobreescriben datos existentes hasta confirmar la operación, garantizando consistencia. APFS soporta snapshots y clones de archivos con costo prácticamente nulo (no duplica datos inmediatamente), cifrado multiclave nativo (por archivo, por usuario o por volumen completo) y está optimizado para almacenamiento de estado sólido mediante comandos TRIM automáticos.

Sistema Operativo	Mecanismo de Gestión RAM	Swap / Archivo de Paginación	Sistema de Archivos Principal	Característica Destacada
Windows 11/10	Paginación bajo demanda, Working Set, SuperFetch	pagefile.sys (ajuste automático)	NTFS	MFT central, ACL, EFS
Linux (kernel 6.x)	Buddy System, Slab Allocator, Page Cache agresivo	Partición o archivo swap, zswap, zram	ext4 / btrfs / XFS	CoW (btrfs), OOM Killer, swappiness
macOS (Ventura/Sonoma)	Unified Memory Architecture (M-series), Memory Compressor	Archivo swap en SSD (último recurso)	APFS	Memoria unificada CPU+GPU, cifrado multiclave

Tabla 2. Comparativa de gestión de memoria y sistemas de archivos en sistemas operativos de escritorio. Cada sistema tiene un enfoque distinto, aunque todos comparten el principio de paginación virtual.

En dispositivos móviles

Los sistemas operativos móviles enfrentan restricciones que no existen en las computadoras de escritorio: la batería es limitada, la RAM está soldada a la placa y no es ampliable, y el almacenamiento flash tiene un número finito de ciclos de escritura. Esto obliga a estrategias de gestión de memoria radicalmente distintas.

La diferencia conceptual más importante es la siguiente: mientras que una computadora de escritorio usa swap en disco cuando se agota la RAM, los móviles prefieren terminar o suspender procesos en segundo plano, porque el acceso frecuente al almacenamiento flash consume energía y degrada el hardware con el tiempo.

Android

Android está construido sobre el kernel de Linux, pero añade capas propias de gestión de memoria adaptadas al entorno móvil. El componente central es el Low Memory Killer (LMK), un módulo del kernel que clasifica los procesos en cinco niveles de prioridad según su visibilidad e importancia para el usuario:

- Foreground (primer plano): la app con la que el usuario interactúa actualmente. Nunca se termina salvo fallo crítico.
- Visible: apps visibles pero sin foco (como un diálogo que cubre parcialmente otra app).
- Service: servicios en segundo plano con trabajo en curso (sincronización, música).
- Cached/Background: apps pausadas que el usuario no usa activamente.
- Empty: procesos sin actividad, mantenidos en memoria solo para relanzarlos más rápido.

Cuando la presión de memoria aumenta, el LMK termina procesos comenzando por los de prioridad más baja (Empty primero, luego Cached, y así sucesivamente). Los procesos terminados pueden ser relanzados rápidamente gracias al proceso Zygote: un proceso padre siempre activo que tiene precargado el entorno de ejecución de Android (ART/Dalvik), de modo que crear una nueva instancia de app es mucho más rápido que arrancarla desde cero.

A partir de Android 10, el LMK fue reemplazado por LMKD (Low Memory Killer Daemon), que corre en espacio de usuario en lugar de espacio de kernel, lo que permite mayor flexibilidad de configuración y mejor integración con cgroups.

zRAM: En lugar de hacer swap tradicional al almacenamiento, Android utiliza zRAM: un dispositivo de bloque en memoria que comprime páginas inactivas dentro de la misma RAM. Con ratios de compresión de 2:1 a 3:1, permite almacenar el equivalente a 6 GB de datos en solo 2–3 GB de RAM real. Esto evita el desgaste del almacenamiento NAND y reduce la latencia respecto al swap en disco. A partir de Android 12, zRAM es obligatorio en dispositivos con menos de 6 GB de RAM.

Sistemas de archivos: La partición de datos del usuario se formatea históricamente con ext4, aunque los fabricantes han migrado a F2FS (Flash-Friendly File System) en dispositivos modernos. F2FS fue diseñado específicamente para almacenamiento NAND flash: utiliza una estructura de registro (log-structured) que agrupa las escrituras de manera secuencial, reduciendo las escrituras aleatorias que degradan la vida útil del flash. La partición del sistema

operativo usa EROFS (Enhanced Read-Only File System) desde Android 10, que ofrece alta compresión y acceso de solo lectura, protegiendo la integridad del sistema.

iOS (Apple)

iOS también está basado en un kernel derivado de BSD (XNU), pero su enfoque hacia la gestión de memoria es más restrictivo y controlado que Android. La política fundamental de iOS es que las apps de tercer nivel no pueden ejecutar código arbitrario en segundo plano de manera indefinida; en cambio, cuando una app pasa al fondo, iOS la suspende (congela su ejecución) o la termina, según la presión de memoria.

Jetsam: El componente encargado de la gestión de memoria bajo presión en iOS es Jetsam (análogo al OOM Killer de Linux, pero más agresivo y silencioso). Cuando la presión de memoria aumenta, Jetsam termina apps en segundo plano sin notificación visible al usuario. Las apps deben implementar el método applicationDidReceiveMemoryWarning() para liberar recursos no esenciales cuando reciben una advertencia de memoria baja.

Ausencia de swap: A diferencia de macOS y Android, iOS nunca ha implementado swap tradicional hacia almacenamiento. Apple justifica esto por tres razones técnicas: (1) el almacenamiento NAND tiene un número limitado de ciclos de escritura, y el swap los agotaría prematuramente; (2) el swap genera latencia perceptible en la interfaz de usuario; (3) iOS ya emplea Memory Compressor (como macOS) para comprimir páginas inactivas dentro de la propia RAM antes de considerar cualquier alternativa. En equipos con chip A-series o M-series, la memoria unificada de alta velocidad reduce aún más la necesidad de mecanismos de extensión.

Sistema de archivos — APFS: Desde iOS 10.3 (2017), todos los dispositivos Apple usan APFS como sistema de archivos. En el contexto móvil, APFS ofrece cifrado por archivo (cada fichero puede tener su propia clave criptográfica, gestionada por el Secure Enclave), snapshots del sistema que permiten actualizaciones OTA (Over-The-Air) sin riesgo de corrupción si se interrumpe la descarga, y uso compartido de espacio libre entre volúmenes del mismo contenedor.

HarmonyOS (Huawei)

HarmonyOS es el sistema operativo desarrollado por Huawei como respuesta a las restricciones comerciales que le impidieron usar Android con servicios de Google. Está diseñado como un sistema operativo distribuido: un dispositivo no es una unidad aislada sino un nodo dentro de un ecosistema que puede incluir teléfonos, tabletas, televisores inteligentes, relojes y automóviles Huawei.

Arquitectura de microkernel: A diferencia de Android (que usa el kernel monolítico de Linux) e iOS (XNU, de arquitectura híbrida), HarmonyOS emplea un microkernel. En esta arquitectura, la mayor parte de los servicios del sistema operativo corren en espacio de usuario como procesos independientes, en lugar de estar integrados en el kernel. Esto aumenta el aislamiento entre servicios: un fallo en el controlador de pantalla, por ejemplo, no necesariamente colapsa el sistema completo, sino que puede reiniciarse de forma independiente.

Memoria distribuida: La característica más innovadora de HarmonyOS en términos de memoria es su capacidad de usar recursos de múltiples dispositivos como si fueran uno solo. Un teléfono puede delegar tareas de procesamiento pesadas a una tableta cercana, utilizando su RAM y CPU de forma transparente para el usuario. Esto se gestiona mediante el bus de hardware distribuido SoftBus, que crea canales de comunicación de baja latencia entre dispositivos del ecosistema Harmony.

Gestión de memoria con IA predictiva: HarmonyOS incluye un módulo de predicción basado en inteligencia artificial que analiza los patrones de uso del usuario (qué apps abre, a qué horas, en qué secuencia) y precarga en memoria las apps que predice que serán necesarias. Esto reduce el tiempo de lanzamiento percibido. Combinado con una política de kill más agresiva para procesos en segundo plano que no han sido usados recientemente, el resultado es una experiencia de usuario fluida incluso con poca RAM disponible.

Sistemas de archivos: HarmonyOS utiliza EROFS para la partición del sistema (garantizando integridad e inmutabilidad del SO) y F2FS para los datos del usuario. Para el ecosistema distribuido, Huawei implementa su propio sistema de archivos virtual distribuido que permite acceder a ficheros de otros dispositivos del mismo ecosistema como si fueran locales.

Característica	Android	iOS	HarmonyOS	Observación
Gestión bajo presión	LMK / LMKD (por prioridad)	Jetsam (suspensión y kill)	Kill por IA + predicción	Todos priorizan la app en primer plano
Swap / Extensión RAM	zRAM (compresión en RAM)	Sin swap; Memory Compressor	zRAM + IA predictiva	Ninguno hace swap a almacenamiento (por defecto)
Sistema de archivos	F2FS (datos), EROFS (sistema)	APFS	F2FS (datos), EROFS (sistema)	APFS destaca por cifrado multiclave y snapshots OTA

Memoria distribuida	No	No	Sí (SoftBus entre dispositivos)	Diferenciador clave de HarmonyOS
Arquitectura del kernel	Linux monolítico	XNU (híbrido Mach+BSD)	Microkernel propio	Afecta el aislamiento y la estabilidad ante fallos

Tabla 3. Comparativa de gestión de memoria en sistemas operativos móviles. Los tres sistemas evitan el swap al almacenamiento, aunque difieren significativamente en su arquitectura y enfoque.

Sistemas operativos de red

Los sistemas operativos de red enfrentan un conjunto de desafíos radicalmente distintos a los de escritorio o móvil. En un servidor de producción pueden ejecutarse simultáneamente cientos o miles de procesos de distintos usuarios, servicios y máquinas virtuales. El objetivo ya no es la responsividad de la interfaz gráfica sino la estabilidad, el aislamiento entre usuarios y el máximo aprovechamiento de recursos compartidos.

Windows Server

Windows Server comparte el núcleo (ntoskrnl.exe) con Windows de escritorio, pero incluye extensiones específicas para entornos empresariales. En arquitecturas de servidor multi-socket, Windows Server implementa soporte NUMA (Non-Uniform Memory Access): cuando un servidor tiene dos o más CPUs físicos, cada uno tiene acceso preferencial a su propio banco de RAM. Acceder a la RAM del banco local es 2–4 veces más rápido que acceder al banco del CPU vecino. El planificador de Windows Server asigna procesos y su memoria preferentemente al mismo nodo NUMA para minimizar la latencia.

Dynamic Memory en Hyper-V: Cuando Windows Server se utiliza como host de virtualización con Hyper-V, la característica Dynamic Memory permite asignar RAM de forma flexible a las máquinas virtuales según su demanda en tiempo real, en lugar de reservar un bloque fijo desde el inicio. Si una VM no está bajo carga, libera su memoria para que otras VMs o el host puedan usarla.

System Cache y SuperFetch/SysMain: El System Cache de Windows puede crecer hasta ocupar prácticamente toda la RAM libre, almacenando metadatos del sistema de archivos NTFS, bloques de archivos frecuentes y datos de aplicaciones. El servicio SysMain (anteriormente SuperFetch) analiza patrones de uso y precarga en memoria los datos que predice que serán necesarios, reduciendo el tiempo de inicio de aplicaciones y servicios.

Sistemas de archivos — NTFS y ReFS: Windows Server usa NTFS como sistema de archivos estándar, pero introduce ReFS (Resilient File System) para escenarios de alta disponibilidad. ReFS calcula checksums automáticos de todos los datos almacenados y los verifica en cada lectura; si detecta corrupción, puede corregirla automáticamente si los datos están en un pool de Storage Spaces con redundancia. ReFS también elimina la necesidad de desfragmentación y soporta volúmenes de hasta 35 PB.

Linux Server (Ubuntu Server, RHEL, Debian)

Linux Server es el sistema operativo dominante en centros de datos a nivel mundial. Su enfoque hacia la gestión de memoria en entornos multiusuario se basa en dos pilares técnicos: cgroups (Control Groups) y namespaces.

cgroups v2: Los Control Groups son una característica del kernel de Linux que permite limitar, contabilizar y aislar el uso de recursos (CPU, memoria, E/S) de grupos de procesos. Mediante cgroups v2, un administrador puede establecer que el servicio MySQL no use más de 8 GB de RAM, o que todos los procesos de un usuario específico estén limitados a 4 GB. Si un proceso supera su límite de memoria asignado, recibe un OOM Kill focalizado que no afecta al resto del sistema. Esta es la tecnología base sobre la que se construyen Docker y Kubernetes.

NUMA en servidores multi-socket: Al igual que Windows Server, Linux implementa soporte NUMA avanzado. La herramienta numactl permite fijar un proceso a un conjunto específico de núcleos y bancos de memoria NUMA. Para bases de datos de alto rendimiento como PostgreSQL o MariaDB, la afinidad NUMA puede representar una mejora del 20–40% en throughput. El kernel usa la política local_alloc por defecto: intenta asignar páginas de memoria del nodo NUMA más cercano al CPU que ejecuta el proceso.

Page Cache masivo: En producción, es habitual que un servidor Linux destine el 70–85% de la RAM disponible al Page Cache. Esto no es un error ni un problema: Linux interpreta la RAM libre como RAM desperdiciada. Cuando un proceso solicita más memoria, el kernel desaloja las páginas menos recientemente usadas del Page Cache para satisfacer la solicitud.

Sistemas de archivos — XFS, ZFS y NFS: XFS es el sistema de archivos predeterminado en Red Hat Enterprise Linux (RHEL) para uso en producción, destacando por su excelente rendimiento en archivos de gran tamaño y alta concurrencia. ZFS (originalmente de Sun Solaris, disponible en Linux a través de OpenZFS) aporta integridad garantizada mediante checksums de datos y metadatos, RAID-Z integrado, snapshots eficientes y deduplicación de datos. Para entornos de red, NFS (Network File System) y SMB/CIFS permiten montar sistemas de archivos remotos como si fueran locales, con su propia capa de caché distribuida.

Aspecto	Windows Server	Linux Server
Aislamiento multiusuario	Sesiones de Terminal Services, cuotas de recursos	cgroups v2 + namespaces por proceso/contenedor
Soporte NUMA	Sí, con afinidad automática en Hyper-V y SQL Server	Sí, configurable con numactl; mejora 20–40% en BD
Caché de disco	System Cache dinámico + SysMain predictivo	Page Cache masivo (70–85% RAM libre)
Sistema de archivos principal	NTFS (estándar), ReFS (alta disponibilidad)	XFS (RHEL), ZFS (OpenZFS), btrfs
Contenedores/VMs	Hyper-V con Dynamic Memory	Docker/Kubernetes sobre cgroups v2

Tabla 4. Comparativa entre Windows Server y Linux Server en aspectos clave de gestión de memoria para entornos de red. Ambos soportan NUMA y aislamiento de recursos, pero difieren en los mecanismos específicos.

Importancia de administrador de memoria

La memoria principal es el recurso más crítico que administra un sistema operativo. Todo proceso en ejecución requiere memoria: sus instrucciones, sus variables, su pila de llamadas y los datos que procesa deben estar en RAM para que el CPU pueda trabajar con ellos. Una gestión deficiente de este recurso puede generar desde simples ralentizaciones hasta vulnerabilidades de seguridad críticas o la caída total del sistema.

Fundamentos de la Importancia

Aislamiento y seguridad entre procesos: Cada proceso debe operar dentro de su propio espacio de direcciones virtuales, sin poder leer ni modificar la memoria de otros procesos. Sin este aislamiento, cualquier aplicación podría leer las contraseñas almacenadas en el navegador, los datos bancarios en otra app o incluso la memoria del kernel. El mecanismo técnico que garantiza este aislamiento es la paginación con tablas de páginas separadas por proceso, respaldada por hardware mediante la MMU (Memory Management Unit).

Rendimiento del sistema: La velocidad de un sistema informático depende en gran medida de cuántos accesos a memoria pueden ser satisfechos desde la caché (rápida) versus desde la RAM o el disco (lentos). Un buen gestor de memoria mantiene en las memorias rápidas los datos más frecuentemente utilizados, aplicando algoritmos como LRU (Least Recently Used). Una caché L1 bien aprovechada puede acelerar los accesos hasta 100 veces respecto a la RAM.

Multitarea eficiente: En un sistema moderno se ejecutan simultáneamente decenas o cientos de procesos. El SO debe distribuir la RAM disponible entre todos ellos de manera justa, garantizando que los procesos críticos (el kernel, los drivers) siempre tengan memoria disponible y que ningún proceso de usuario pueda acaparar todos los recursos.

Estabilidad del sistema: Una administración correcta previene condiciones críticas como el agotamiento total de memoria (OOM), el thrashing o los deadlocks por recursos. Cuando el sistema detecta que la disponibilidad de memoria cae por debajo de un umbral crítico, debe actuar proactivamente (terminando procesos de baja prioridad, comprimiendo páginas, liberando caché) antes de que el sistema llegue a un estado irrecuperable.

Consecuencias de una Mala Gestión de Memoria

Los efectos de una administración deficiente de memoria son variados y pueden ir desde inconvenientes menores hasta fallos catastróficos:

Consecuencia	Descripción Técnica	Ejemplo Real
Crash del sistema	Agotamiento total de la RAM sin mecanismo de recuperación	Android sin LMK activo: el kernel entra en pánico (kernel panic)
Degradación del rendimiento (thrashing)	El sistema invierte más tiempo en swap que en ejecución real	Windows con 4 GB RAM y 30 tabs de Chrome: el disco trabaja al 100% constantemente
Vulnerabilidades de seguridad	Acceso a memoria de otro proceso por desbordamiento o lectura sin validación	Heartbleed (2014): bug en OpenSSL que permitía leer 64 KB de RAM del servidor sin autenticación
Fugas de memoria	Procesos que asignan memoria y nunca la liberan	Firefox antiguo requería reiniciarse cada días; servidores de producción que crecen indefinidamente
Denegación de servicio	Un proceso malicioso consume toda la RAM disponible	Fork bomb en Linux sin cgroups: el sistema deja de responder completamente
Corrupción de datos	Escrituras en zonas de memoria no autorizadas sobreescriben estructuras críticas	BSOD (Pantalla Azul) en Windows causado por drivers con bugs de escritura en memoria del kernel

Tabla 5. Consecuencias de una gestión deficiente de memoria. Cada falla tiene un impacto específico que puede afectar desde el rendimiento hasta la seguridad del sistema.

Llevado de control de los espacios disponibles y ocupados

Para administrar la memoria de forma eficiente, un sistema operativo debe conocer en todo momento qué regiones de la memoria están libres y cuáles están asignadas, a quién pertenecen y qué permisos tienen. Esto requiere estructuras de datos especializadas, algoritmos de asignación y mecanismos de traducción de direcciones implementados tanto en software (el kernel) como en hardware (la MMU).

Representación del Espacio Libre y Ocupado

Mapas de bits (bitmap): La memoria se divide en unidades de tamaño fijo (páginas o marcos). Cada unidad tiene un bit en el mapa: 0 si está libre, 1 si está ocupada. Es simple pero lento para encontrar regiones libres contiguas de gran tamaño.

Listas ligadas de segmentos: Se mantiene una lista de segmentos, cada uno con información sobre si está libre u ocupado, su dirección inicial y su tamaño. Cuando se libera un bloque, se fusiona con los vecinos libres adyacentes para reducir la fragmentación.

Buddy System (sistema compañero): Divide la memoria en bloques de potencias de dos. Al liberar un bloque, busca su 'compañero' (buddy) del mismo tamaño adyacente; si también está libre, los fusiona en un bloque del doble de tamaño. Este proceso se repite de forma recursiva. Es el algoritmo que usa el kernel de Linux para la asignación de páginas físicas.

Asignación de Memoria

Cuando un proceso solicita memoria, el gestor puede aplicar distintas estrategias para encontrar un bloque libre adecuado:

- First-fit (primer ajuste): asigna el primer bloque libre suficientemente grande. Rápido pero deja fragmentos al inicio de la memoria.
- Best-fit (mejor ajuste): busca el bloque libre más pequeño que satisfaga la solicitud. Minimiza el desperdicio por bloque pero genera muchos fragmentos pequeños.
- Worst-fit (peor ajuste): asigna el bloque libre más grande disponible. La idea es dejar fragmentos grandes que puedan ser útiles, pero en la práctica genera mayor fragmentación.
- Next-fit (siguiente ajuste): variante de First-fit que comienza la búsqueda desde donde terminó la última asignación, distribuyendo los fragmentos más uniformemente.

Paginación

La paginación es el mecanismo más importante en los sistemas operativos modernos para gestionar el espacio de memoria. Consiste en dividir tanto la memoria física (en marcos o frames de tamaño fijo, típicamente 4 KB) como el espacio de direcciones virtual de cada proceso (en páginas del mismo tamaño). Cada proceso tiene su propia tabla de páginas que mapea páginas virtuales a marcos físicos.

La traducción de direcciones ocurre de la siguiente manera: cuando el CPU genera una dirección virtual, la MMU la divide en dos partes: el número de página virtual y el desplazamiento (offset) dentro de esa página. El número de página se usa como índice en la tabla de páginas del proceso para obtener el número de marco físico correspondiente. La dirección física final se calcula combinando el número de marco con el offset original.

Para acelerar este proceso, la MMU incluye una memoria caché especializada denominada TLB (Translation Lookaside Buffer), que almacena las traducciones más recientes. Si la dirección virtual ya está en el TLB (TLB hit), la traducción toma solo 1–2 ciclos; si no (TLB miss), debe consultarse la tabla de páginas en RAM, lo que toma decenas de ciclos adicionales.

Ejemplo ilustrativo: Un proceso accede a la dirección virtual 0x8004 con páginas de 4 KB. El número de página es 0x8004 / 4096 = página 8; el offset es 0x8004 % 4096 = 4. La tabla de páginas indica que la página 8 corresponde al marco físico 23. La dirección física es entonces 23 × 4096 + 4 = 0x17004. Todo este proceso lo realiza la MMU en hardware, de forma transparente al proceso y al programador.

Segmentación

La segmentación divide el espacio de direcciones en segmentos de tamaño variable con significado lógico: un segmento para el código del programa, otro para los datos globales, otro para la pila y así sucesivamente. A diferencia de la paginación, la segmentación es visible para el programador y refleja la estructura lógica del programa.

La arquitectura x86 históricamente usó segmentación combinada con paginación, aunque en los sistemas de 64 bits (modo long de x86-64) la segmentación está prácticamente deshabilitada y toda la protección recae en la paginación. En sistemas embebidos y microcontroladores sin MMU, la segmentación sigue siendo el mecanismo principal de separación de áreas de memoria.

Liberación y Algoritmos de Reemplazo de Páginas

La liberación de memoria puede ser explícita (el programador llama a free() o delete) o implícita (a través de un recolector de basura en lenguajes como Java, Python o C#, o del modelo de ownership de Rust). Cuando el programa retorna de una función, el marco de pila (stack frame) se libera automáticamente por el ABI (Application Binary Interface). Cuando un proceso termina, el kernel recupera todas sus páginas sin excepción.

Cuando la RAM está llena y se necesita cargar una nueva página del disco, el sistema operativo debe decidir qué página existente desalojar. Los principales algoritmos son:

Algoritmo	Criterio de Reemplazo	Ventaja	Desventaja
FIFO	La página más antigua en memoria	Sencillo de implementar	Puede eliminar páginas muy usadas (anomalía de Bélády)
LRU	La página menos recientemente usada	Muy eficiente en la práctica	Costoso de implementar perfectamente en hardware
LFU	La de menor frecuencia de uso histórico	Considera el uso a largo plazo	Páginas antiguas muy usadas permanecen, desplazando nuevas
Clock (Segunda oportunidad)	Aproximación LRU usando bit de referencia en un buffer circular	Bajo costo computacional, muy práctico	Es una aproximación; no siempre óptimo
Óptimo (OPT)	La página que no se usará durante más tiempo en el futuro	Mínimo número de fallos de página posible (teórico)	Irrealizable en la práctica (requiere conocer el futuro)

Tabla 6. Comparativa de algoritmos de reemplazo de páginas. En la práctica, Linux y Windows usan variantes del algoritmo Clock (también llamado segunda oportunidad), que aproxima LRU con bajo costo computacional.

Problemas más frecuentes en la administración de memoria

A pesar de la sofisticación de los gestores de memoria modernos, existen problemas recurrentes que afectan tanto al diseño de sistemas operativos como al desarrollo de software de aplicación. Comprender estos problemas es esencial para cualquier ingeniero en sistemas.

Fragmentación

La fragmentación es la condición en la que la memoria disponible existe pero no puede ser utilizada eficientemente debido a su distribución no óptima. Se presenta en dos formas:

Fragmentación externa: Se produce cuando hay suficiente memoria libre en total, pero no en un bloque contiguo suficientemente grande para satisfacer una solicitud. Imaginemos que un disco tiene 500 MB libres distribuidos en 100 fragmentos de 5 MB cada uno, pero un proceso necesita un bloque contiguo de 50 MB: la solicitud fallará aunque la memoria libre total sea diez veces mayor. Este problema es especialmente relevante en sistemas con asignación de memoria no paginada.

Fragmentación interna: Ocurre cuando se asigna más memoria de la que el proceso realmente necesita, como consecuencia de trabajar con bloques de tamaño fijo. Si el sistema asigna bloques de 4 KB y un proceso solo necesita 100 bytes, se desperdician 3,996 bytes por cada bloque asignado. La paginación elimina la fragmentación externa pero introduce fragmentación interna en la última página asignada a cada proceso.

La paginación resuelve la fragmentación externa porque cualquier marco físico libre puede satisfacer cualquier solicitud de página, independientemente de su ubicación en la memoria física. El Slab Allocator de Linux mitiga la fragmentación interna para objetos del kernel al mantener cachés de objetos del tamaño exacto necesario.

Fuga de Memoria (Memory Leak)

Una fuga de memoria ocurre cuando un proceso asigna memoria dinámicamente pero nunca la libera, aunque ya no la necesita. La memoria queda en un estado de limbo: el sistema operativo la considera ocupada (porque un proceso la solicitó), pero el proceso tampoco la usa. Con el tiempo, la memoria disponible del sistema se agota progresivamente.

Ejemplo en C: En el lenguaje C, la función malloc() reserva memoria del heap. Si el programador no llama a free() para liberarla, la memoria no se recupera hasta que el proceso

termine. Un servidor web en C que atiende millones de peticiones y tiene una fuga de solo 100 bytes por petición perderá 100 MB de RAM por cada millón de peticiones.

Las fugas de memoria en lenguajes con recolección de basura (Java, Python, C#) son más sutiles: ocurren cuando el código mantiene referencias a objetos innecesarios, impidiendo que el GC los elimine. Por ejemplo, agregar objetos a una colección sin eliminarlos cuando ya no se necesitan.

Las herramientas de detección incluyen Valgrind en Linux, AddressSanitizer (ASan) integrado en GCC y Clang, el perfil de memoria de Instruments en macOS, y Memory Profiler en herramientas de Visual Studio para Windows.

Thrashing (Hiperpaginación)

El thrashing es una condición crítica de rendimiento que ocurre cuando el sistema tiene demasiados procesos compitiendo por una cantidad insuficiente de RAM. En esta situación, las páginas de memoria están siendo constantemente movidas entre la RAM y el disco (swap): en cuanto se carga una página para un proceso, es necesario desalojarla para cargar otra página para un proceso diferente.

El resultado es que el CPU parece estar al 100% de uso, pero prácticamente todo ese tiempo se invierte en esperar operaciones de E/S del disco de swap, no en trabajo productivo. El sistema se vuelve virtualmente inutilizable: la barra de progreso avanza lentísimo, el disco gira sin parar y el tiempo de respuesta se mide en minutos en lugar de millisegundos.

Ejemplo ilustrativo: Una computadora con 4 GB de RAM está ejecutando un navegador con 40 pestañas (usando 3 GB), una suite de ofimática (1 GB) y un editor de video (2 GB). Los 6 GB requeridos superan en 2 GB la RAM disponible. El sistema entra en thrashing: cuando el editor de video necesita sus páginas, desaloja páginas del navegador al swap; cuando se vuelve al navegador, las páginas del editor se van al swap; y así indefinidamente.

La solución técnica es el modelo de Working Set: el SO monitoriza el conjunto de páginas que cada proceso ha usado en un intervalo de tiempo reciente. Si la suma de todos los working sets supera la RAM disponible, el SO reduce el grado de multiprogramación (suspende algunos procesos) hasta que la situación se estabilice.

Deadlock por Recursos de Memoria

Un deadlock (interbloqueo) de memoria ocurre cuando dos o más procesos esperan indefinidamente recursos de memoria que el otro proceso retiene y no liberará. Es una generalización del deadlock de recursos hacia el dominio específico de la memoria.

Ejemplo: El Proceso A tiene 300 MB asignados y solicita 200 MB adicionales. El Proceso B tiene 200 MB asignados y solicita 100 MB adicionales. El sistema solo tiene 150 MB libres, insuficientes para satisfacer ninguna de las dos solicitudes. Ambos procesos quedan bloqueados esperando que el otro libere su memoria, pero ninguno puede avanzar para hacerlo.

Las condiciones de Coffman (necesarias para que exista un deadlock) aplicadas a memoria son: exclusión mutua (cada página solo puede pertenecer a un proceso), retención y espera (el proceso retiene lo que tiene mientras espera más), no apropiación (el SO no puede forzar al proceso a liberar) y espera circular. El algoritmo del banquero de Dijkstra es un mecanismo clásico de prevención que solo permite asignaciones que mantienen el sistema en un estado seguro.

Desbordamiento de Búfer (Buffer Overflow)

Un desbordamiento de búfer ocurre cuando un programa escribe más datos en un buffer de tamaño fijo de los que este puede contener, sobrescribiendo la memoria adyacente. Dependiendo de qué memoria se sobrescriba (variables locales, dirección de retorno de una función, datos del heap), las consecuencias pueden ir desde un simple crash hasta la ejecución de código malicioso arbitrario.

Ejemplo clásico: Una función en C reserva un array local de 100 caracteres para almacenar el nombre de usuario. Si el servidor acepta entrada de red sin validar la longitud y el atacante envía 500 caracteres, los 400 bytes adicionales sobreescriben la pila de llamadas, incluyendo potencialmente la dirección de retorno de la función. El atacante puede hacer que el programa salte a código malicioso que él mismo ha inyectado.

Las mitigaciones modernas incluyen: ASLR (Address Space Layout Randomization), que aleatoriza las direcciones base del código, heap y pila, dificultando que el atacante sepa a dónde saltar; DEP/NX bit (Data Execution Prevention / No-Execute), que impide ejecutar código en regiones de datos; Stack Canaries, valores aleatorios colocados antes de la dirección de retorno que el programa verifica antes de retornar; y el lenguaje Rust, que elimina esta clase de errores por diseño a través de su sistema de tipos y de ownership.

Use-After-Free

El error use-after-free ocurre cuando un programa libera un bloque de memoria con free() y luego intenta acceder a él de nuevo. La memoria liberada puede haber sido reasignada a otro objeto, de modo que la lectura o escritura posterior opera sobre datos inesperados, causando corrupción de datos o vulnerabilidades de seguridad.

Este tipo de error es la base de numerosas vulnerabilidades críticas en navegadores web (motores JavaScript como V8 y SpiderMonkey han tenido múltiples CVEs por use-after-free) y

en el kernel de Linux. AddressSanitizer detecta automáticamente este tipo de error en tiempo de ejecución durante las fases de prueba.

Problema	Causa Raíz	Impacto	Mitigación Moderna
Fragmentación externa	Asignación variable sin compactación	Fallos al asignar bloques grandes	Paginación (la elimina por completo)
Fragmentación interna	Bloques de tamaño fijo mayores que los datos	Desperdicio de RAM	Slab Allocator, tamaños de bloque variables
Memory Leak	Código sin free(), referencias persistentes en GC	Degradación progresiva hasta crash	GC, Rust ownership, Valgrind, ASan
Thrashing	Suma de working sets > RAM disponible	Sistema inutilizable, E/S de disco al 100%	Working Set model, cgroups, LMK, agregar RAM
Deadlock de memoria	Espera circular entre procesos por recursos	Procesos bloqueados indefinidamente	Algoritmo del banquero, timeouts, detección proactiva
Buffer Overflow	Sin validación de límites en escrituras de buffer	Crash, ejecución de código malicioso	ASLR, DEP/NX, Stack Canaries, Rust
Use-After-Free	Acceso a memoria ya liberada	Corrupción de datos, vulnerabilidades CVE	Rust ownership, AddressSanitizer, smart pointers

Tabla 7. Resumen comparativo de los principales problemas en la administración de memoria, sus causas, impactos y las mitigaciones disponibles en sistemas modernos.

Conclusiones individuales

Angely Ximena Cruz Ortega

Esta actividad resultó especialmente interesante para mí, ya que desde pequeña he tenido curiosidad por probar distintos sistemas operativos. Durante la secundaria utilizaba únicamente macOS, el cual hasta la fecha considero muy fácil de usar gracias a su diseño intuitivo y amigable,

especialmente para usuarios que no tienen tanta experiencia con computadoras.

Más adelante, comencé a usar una computadora con Windows, sistema que también me gustó bastante, aunque en ese momento no lograba superar la comodidad que sentía con macOS. Posteriormente descubrí Linux, y mi primera experiencia no fue positiva; aunque me atraía mucho la posibilidad de personalizar el sistema, me resultaba frustrante depender constantemente de la consola, por lo que decidí dejarlo y continuar con Windows.

Sin embargo, al entrar a la preparatoria, mi interés por Linux regresó y mi perspectiva cambió completamente. Comencé a utilizar distribuciones como Fedora y Debian, incluso para jugar, lo cual mejoró mi experiencia en comparación con Windows en ciertos casos. Esto me llevó a considerar Debian como mi sistema principal, aunque encontré algunas limitaciones en herramientas como LibreOffice frente a Microsoft Office.

Con el tiempo, al contar con una mejor computadora, pude explorar Linux con mayor libertad. Actualmente, Linux Mint es mi distribución favorita, la cual utilizo en una máquina virtual, e incluso logré darle nueva vida a una tablet antigua instalando este sistema.

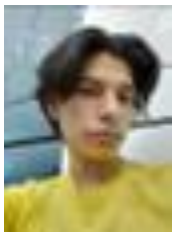
A partir de esta experiencia personal y de la investigación realizada, puedo concluir que no existe un sistema operativo universalmente superior. La elección adecuada depende de factores como el rendimiento, la seguridad, el costo y las necesidades específicas del usuario o del entorno de trabajo. Comprender el funcionamiento interno de estos sistemas permite tomar decisiones más informadas, en lugar de basarse únicamente en preferencias o tendencias.

Conclusión Individual

Esta actividad resultó especialmente interesante para mí, ya que desde pequeño he tenido curiosidad por probar distintos SO. Durante la pandemia utilizaba solo macOS, el cual hasta la fecha considero muy fácil de usar gracias a su diseño intuitivo y amigable, especialmente para usuarios que no tienen tanta experiencia con computadoras.
Más adelante, comencé a usar una computadora con Windows, sistema que también me gustó bastante, aunque en ese momento no lograba superar la comodidad que sentía con macOS. Posteriormente descubrí Linux, y mi primera experiencia no fue muy positiva, aunque me atraía mucho la posibilidad de personalizar el sistema, me resultaba frustrante depender constantemente de la consola, por lo que decidí dejarlo y continuar con Windows
Sin embargo, al entrar a la preparatoria, mi interés por linux regresó y mi perspectiva cambió completamente. Comencé a utilizar distribuciones como Fedora y Debian, incluso para jugar, lo cual mejoró mi experiencia en comparación con Windows en ciertos casos. Esto me llevó a considerar Debian como mi sistema principal, aunque encontré muchas limitaciones en herramientas como LibreOffice frente a Microsoft Office
A partir de esta experiencia personal y de la investigación realizada, puedo concluir que no existe un sistema operativo universalmente superior. La elección depende de factores como el rendimiento, la seguridad, el costo y las necesidades específicas del usuario o del entorno de trabajo. Comprender el funcionamiento interno de estos sistemas permite tomar decisiones más informadas, en lugar de basarse únicamente en preferencias o tendencias

CS Escaneado con CamScanner

Ari Sebastián García Martínez



Después de analizar todo esto sobre memorias y sistemas de archivos, me doy cuenta de que es mucho más complejo de lo que pensaba. Basicamante, los dispositivos como computadoras, celulares y tablets tienen diferentes formas de guardar información dependiendo del sistema operativo que usen. Lo que más me sorprendió es que cada sistema operativo maneja la memoria de manera tan distinta, y no es solo tener espacio disponible, sino que el sistema tiene que estar constantemente controlando qué espacios estan ocupados y cuales están libres, como si fuera una biblioteca gigante donde alguien tiene que saber exactamente donde está cada libro. Por eso entendi que la administración de memoria es importante, porque sin ella los dispositivos se volverían lentos o dejarían de funcionar, y ahora entiendo porque mi celular a veces se traba.

Conclusion - Ari Sebastian Garcia Martinez

Despues de analizar todo esto sobre memorias y sistemas de archivos, me doy cuenta de que es mucho mas complejo de lo que pensaba. Basicamente, los dispositivos como computadoras, celulares y tablets tienen diferentes formas de guardar informacion dependiendo del sistema operativo que usen. Lo que mas me sorprendio es que cada sistema operativo maneja la memoria de manera tan distinta, y no es solo tener espacio disponible, sino que el sistema tiene que estar constantemente controlado que espacio estan ocupados y cuales estan libres, como si fuera una biblioteca gigante donde alguien tiene que saber exactamente donde esta cada libro.

Scanned with CamScanner

Diego Alonso Pérez Barrón

En resumen el almacenaje la memoria y los archivos son como el cuerpo y la mente de tu compu la memoria RAM es como la mesa de trabajo es rápida y ahí tienes lo que estás usando en este momento pero si apagas la máquina se vacía todo el almacenamiento ya sea disco duro o SSD es como el armario o la caja fuerte ahí guardas todo lo que quieres conservar a largo plazo tus fotos documentos juegos y se queda ahí aunque apagues la compu y los archivos son justamente las cosas que guardas son como las hojas de papel las fotos o los videos que metes en ese armario para que tu compu funcione bien necesitas que la RAM sea suficiente para que no se trabe cuando haces varias cosas a la vez y que el almacenamiento tenga espacio suficiente para guardar todo lo que te importa son tres cosas que van de la mano para que tu equipo sea tu mejor aliado

23/03/2026

Diego Alonso Pérez Barrón 2099212 ITS
Sistemas operativos Grupo=01S Salon=4208

En resumen el almacenaje la memoria y los archivos son como el cuerpo y la mente de tu compu la memoria RAM es como la mesa de trabajo es rápida y ahí tienes lo que estas usando en este momento pero si apagas la máquina se vacía todo el almacenamiento ya sea disco duro o SSD es como el armario o la caja fuerte ahí guardas todo lo que quieres conservar a largo plazo tus fotos documentos juegos y se queda ahí aunque apagues la compu y los archivos son justamente las cosas que guardas son como las hojas de papel las fotos o los videos que metes en ese armario para que tu compu funcione bien necesitas que la RAM sea suficiente para que no se trabe cuando hace varias cosas a la vez y que el almacenamiento tenga espacio suficiente para guardar todo lo que te importa son tres cosas que van de la mano para que tu equipo sea tu mejor aliado.

Referencias

Apple. (2019). *Apple Developer*. Apple.com. https://developer.apple.com/

Desarrolladores de Android. (n.d.). Android Developers. https://developer.android.com/?hl=es-419

Linux Kernel Documentation. (n.d.). Www.kernel.org. https://www.kernel.org/doc/

Stallings, W. (2018). *Operating Systems: internals and design principles* (9th ed.). Pearson Education Limited.

wibjorn. (2021). *Microsoft Learn: desarrolle habilidades que le abran puertas en su carrera*. Microsoft.com. https://learn.microsoft.com/es-mx/